

Local Clang Setup

I'm working on a project: <https://github.com/tslator/safety-critical-high-availability>. Principally, GCC 13 is the compiler, but it is becoming apparent that being able to run CLang for verification and checking is very useful. I could just install clang globally, but I'm wondering if it makes sense to keep the clang install local to the project and 'tagged' as for its intended purpose, checking, verification, validation. If so, what is the best way to do this and make the intended purpose clear?

RouteLLM

Recommendation

Yes—keep the Clang *checking toolchain* project-scoped, but **do not vendor a Clang installation into the repository** and do not rely on an unversioned host-global installation as the project's authoritative checker.

For this repository, the cleanest approach is a **dedicated, pinned verification container image** plus a named CMake preset such as clang-verify.

That gives you:

- A distinct and documented role: GCC is the primary build compiler; Clang is an independent analysis/checking compiler.
- Reproducible tool versions across developers and CI.
- No host-toolchain drift.
- Separate build trees, preventing CMake from mixing GCC and Clang object files or cached compiler settings.
- Easy collection of evidence: image digest, compiler version, configuration command, test results, static-analysis findings.

Your existing Dockerfile already establishes a good precedent: a digest-pinned Debian image and containerized build/test baseline. Extend that pattern rather than installing into /opt under the checkout.

One immediate item to resolve: although the project intent is GCC 13, the current Dockerfile installs build-essential rather than explicitly requesting gcc-13 and g++-13.

Verify the actual compiler in that image and make the primary compiler selection explicit before treating its results as baseline evidence.

Suggested terminology

Use these terms consistently in documentation, directory names, CMake presets, image tags, and CI job names:

Purpose	Suggested name
Primary production/compiler baseline	gcc-primary
Independent compile-and-test check	clang-verify
Clang static analysis	clang-static-analysis
Dynamic memory/runtime instrumentation	clang-sanitizers
Reproducible container	verification-toolchain

I would avoid calling a successful Clang build “validation.” It is useful **supplementary verification evidence**, but it does not validate the system’s safety requirements by itself.

Repository structure

A simple, intention-revealing structure:

```
CMake/  
  toolchains/  
    clang-verify.cmake  
  presets/  
    CMakePresets.json  
containers/  
  verification/  
    Dockerfile.clang  
docs/  
  verification/  
    clang-verification.md
```

If you prefer to keep the repository’s current flat Docker layout, Dockerfile.clang-verify at the root is also reasonable. The important part is the **role in its name**.

CMake toolchain file

CMake/toolchains/clang-verify.cmake:

```

# Clang is a supplementary verification compiler.
# It is not the project's primary production compiler.

set(CMAKE_C_COMPILER clang CACHE FILEPATH "")
set(CMAKE_CXX_COMPILER clang++ CACHE FILEPATH "")

set(CMAKE_CXX_STANDARD 20 CACHE STRING "")
set(CMAKE_CXX_STANDARD_REQUIRED ON CACHE BOOL "")
set(CMAKE_CXX_EXTENSIONS OFF CACHE BOOL "")

add_compile_options(
  -Wall
  -Wextra
  -Wpedantic
  -Wconversion
  -Wsign-conversion
  -Wshadow
  -Wformat=2
)

```

Keep warnings appropriate to your codebase. Do not automatically make every Clang-specific warning fatal until you have reviewed whether each one is a project policy requirement. A warning policy should be intentional, stable, and documented.

CMake preset

Use a dedicated build directory. CMake chooses a compiler during its first configure, so never reconfigure an existing GCC build directory as Clang.

```

{
  "version": 6,
  "configurePresets": [
    {
      "name": "clang-verify",
      "displayName": "Clang supplementary verification",
      "generator": "Ninja",
      "binaryDir": "${sourceDir}/build/clang-verify",
      "toolchainFile": "${sourceDir}/CMake/toolchains/clang-verify.cmake",
      "cacheVariables": {
        "SAFETY_CRIT_BUILD_TESTING": "ON",
        "SAFETY_CRIT_TEST_FRAMEWORK": "GoogleTest"
      }
    }
  ]
}

```

```

    }
  }
],
"buildPresets": [
  {
    "name": "clang-verify",
    "configurePreset": "clang-verify"
  }
],
"testPresets": [
  {
    "name": "clang-verify",
    "configurePreset": "clang-verify",
    "output": {
      "outputOnFailure": true
    }
  }
]
}

```

The resulting developer and CI sequence is explicit:

```

cmake --preset clang-verify
cmake --build --preset clang-verify
ctest --preset clang-verify

```

Verification container

Make the toolchain image a project artifact, tagged by its function and compiler version, for example:

```
safety-critical-ha:verify-clang-<documented-version>
```

The container should install the selected Clang package, clang-tidy, and the Clang analysis tools you actually use. Its build should record the precise versions:

```

RUN clang --version \
&& clang-tidy --version \
&& cmake --version \
&& ninja --version

```

For evidence-grade reproducibility:

1. Pin the base image by digest, as you already do.
2. Select a specific Clang major version rather than relying on whatever clang currently resolves to.
3. Record the package version and package source/repository in the image build log or a generated tool manifest.
4. Use the same image in CI and for the documented local verification command.
5. Rebuild only through an intentional toolchain-update change, not opportunistically on every unrelated build.

A useful separation is:

- Dockerfile: primary build/runtime pipeline.
- containers/verification/Dockerfile.clang: Clang verification pipeline.
- CI job clang-verify: configure, compile, test, and publish diagnostics.
- CI job clang-static-analysis: run clang-tidy or Clang analyzer separately, with its own ruleset and findings policy.

What Clang should do

At minimum, make the Clang job an independent clean build and test run:

```
cmake --preset clang-verify  
cmake --build --preset clang-verify  
ctest --preset clang-verify
```

Then add separate gates incrementally:

1. **Cross-compiler compilation:** GCC primary build versus Clang checking build.
2. **Unit/integration tests:** run against both builds where practical.
3. **Static analysis:** clang-tidy with a checked-in configuration and documented enabled checks.

4. **Sanitizers:** separate build presets for ASan/UBSan; these should not be quietly blended into the ordinary Clang job.
5. **Compiler diagnostics review:** deliberate project policy for warnings, suppressions, and deviations.

Make the intended purpose unambiguous

In docs/verification/clang-verification.md, state something close to:

GCC 13 is the project's primary compiler baseline. The Clang verification toolchain is an independently configured supplementary verification environment. It is used to identify compiler-diagnostic differences, execute the test suite under an alternative implementation, run approved static-analysis checks, and support sanitizer-enabled test executions. A successful Clang result is verification evidence; it does not, by itself, constitute system validation or safety certification.

Also maintain a small evidence table:

Evidence item	Required record
Toolchain identity	Image digest, Clang version, package version
Configuration	CMake preset and cache variables
Build result	Compiler diagnostics and exit status
Test result	CTest output and test count
Static analysis	Ruleset version, findings, approved deviations
Sanitizer run	Enabled sanitizers, test command, findings

Bottom line

Use a **project-owned, version-pinned Clang verification container** and a **separate clang-verify CMake preset/build directory**. Treat Clang as a deliberately independent source of verification evidence—not as an alternate everyday host installation, and not as a substitute for formal validation.
